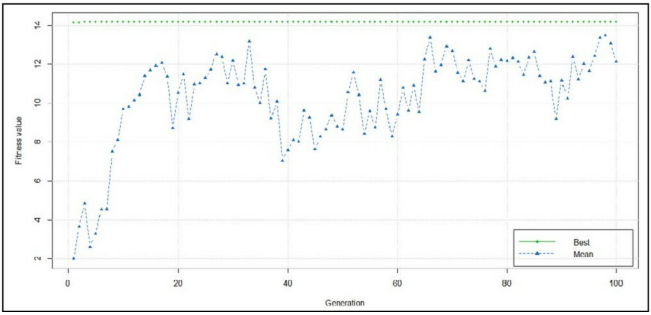

Figure 4: GA

neural network is ready to predict the 61st instance for us, and Statement 7 does exactly that. The deviation from the actual result is due to very few instances to train neurons. As the learning instance increases, the result will converge more accurately. A good read on training neural nets is provided in the following research journal: *http://journal.r-project.org/archive/2010-1/RJournal_2010-1_Guenther+Fritsch.pdf*

Genetic algorithms (GA): This is the class of algorithms that can leverage evolution-based heuristic techniques to solve a problem. GA is represented by a chromosome like data structure, which uses recursive recombination or search techniques. GA is applied over the problem domain in which the outcome is very unpredictable and the process of generating the outcome contains complex inter-related modules. For example, in the case of AIDS, the human immunodeficiency virus becomes resistant to antibiotics after a definite span of time, after which the patient is dosed with a completely new kind of antibiotic. This new antibiotic is prepared by analysing the pattern of the human immunodeficiency virus and its resilient nature. As time passes, finding the required pattern becomes very complex and, hence, leads to inaccuracy. The GA, with its evolutionary-based theory, is a boon in this field. The genes defined by the algorithm generate an evolutionary-based antibiotic for the respective patient. One such case to be mentioned here is IBM's EuResist genomics project in Africa.

The following code begins by installing and loading the GA package into the R environment. We then define a function which will be used for fitting; it contains a summation of the extended sigmoid function and sine function. The fifth statement performs the necessary fitting within maximum and minimum limits. We can then summarise the result and can check its behaviour by plotting it.

```
>install.packages("GA");
>library("GA");
>f <- function(x){ 1/(exp(x)+exp(-x)) + x*sin(x) };
>plot(f,-15,15);
>geneticAl <- ga(type = "real-valued", fitness = f, min = -15,
max = 15);
>summary(geneticAl);
>plot(geneticAl); // Refer to Figure 4
```

A trade off (paradox of exponential degradation): The type of problem where the algorithms defined run in super polynomial time [$O(2^n)$ or $O(n^n)$] rather than polynomial time [$O(n^k)$ for some constant k] are called combinatorial explosion problems or non polynomial. Neural networks and genetic algorithms often fall into the category of non polynomial. Neural networks contain neurons that are complex mathematical models; several thousands of neurons are contained in a single neural net and any practical learning network comprises several hundreds of neural nets, taking the mathematical equations to a very high degree. The higher the degree, the greater is the efficiency. But the higher degree has an adverse effect on computability time (ignoring space complexity). The idea of limiting and approximating results is called amortized analysis. R code does not provide any inbuilt tools to check combinatorial explosion. END

By: Munawar Hasan

The author has been an algorithm developer for more than three-and-a-half years. Recently he developed a predictive algorithm for financial modelling to detect several types of fraud in banking and insurance. He owns a cloud computing framework to facilitate true virtualisation. He has written several research papers and white papers related to computational analysis and cloud simulation.